MARMARA ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

MİKROİŞLEMCİLER DERSİ

PROJE RAPORU

PLATFORM ATLAMA OYUNU (EMU8086)

Hazırlayan : Nesrin Güler

Öğrenci numarası : 170424044

Teslim Tarihi : 27.04.2026

İçindekiler

1. ÖZET 3
2. GİRİŞ 3
3. YÖNTEM 3
3.1 Kullanılan Araçlar 3
3.2 Gerçekleştirilen Çalışmalar 4
4. UYGULAMA 4
4.1 Veri Yapısı 4
4.2 Oyun Döngüsü 5
4.3 Klavye Girişi 5
4.4 Zıplama Mekaniği 6
4.5 Engel Hareketi 7
4.6 Çarpışma Kontrolü 7
4.7 Skor Sistemi 7
4.8 Ekrana Yazdırma 8
4.9 Game Over 8
4.10 Prosedürler ve Fonksiyonel Yapı 9
4.10.1 Zıplama Kontrol Prosedürü 9
4.10.2 Skor Kontrol Prosedürü 9
4.10.3 Çarpışma Kontrol Prosedürü 10
4.10.4 Ekran Çizim Prosedürleri 11
4.10.5 Ekrana Karakter Basma 11
4.10.6 İmleç Konumlandırma 11
4.10.7 Zemin Çizimi 12
5. BULGULAR 12
6. SONUÇ 13
7. ÖNERİLER 13

1. ÖZET

Bu projede EMU8086 ortamında 8086 Assembly dili kullanılarak basit bir platform atlama oyunu geliştirilmiştir. Oyunda kullanıcı, klavye girdisi ile karakteri kontrol ederek hareket eden engellerden kaçmaya çalışmaktadır. Sistem, BIOS interrupt'ları kullanılarak gerçek zamanlı giriş-çıkış işlemleri ile çalışmaktadır.

Proje sonucunda düşük seviyeli programlama, bellek yönetimi ve interrupt mekanizmaları uygulamalı olarak gerçekleştirilmiştir.

2. GİRİŞ

Assembly dili, donanım ile doğrudan iletişim kurmayı sağlayan düşük seviyeli bir programlama dilidir. Bu projede amaç, bu dilin temel yapılarını kullanarak gerçek zamanlı çalışan basit bir oyun geliştirmektir.

Bu kapsamda:

- Klavye girdisi alınması
- Ekrana karakter çizimi
- Oyun döngüsü oluşturulması
- Çarpışma ve skor mekanizması
-

gerçekleştirilmiştir.

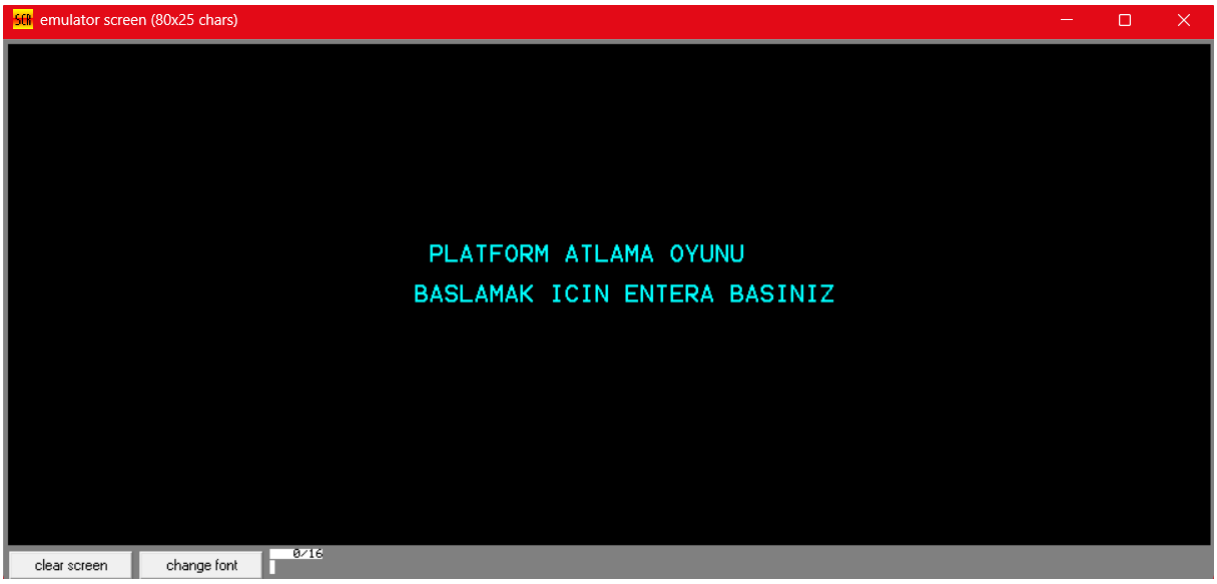
3. YÖNTEM

3.1 Kullanılan Araçlar

- EMU8086
- Assembly (8086)
- BIOS Interrupt (INT 10h, INT 16h, INT 15h)

3.2 Gerçekleştirilen Çalışmalar

- Oyun ekranı oluşturuldu
- Karakter hareket sistemi geliştirildi
- Engel mekanizması kuruldu
- Skor sistemi eklendi
- Çarpışma kontrolü yapıldı
- Game Over ve restart sistemi eklendi



4. UYGULAMA

4.1 Veri Yapısı

Aşağıdaki değişkenler oyun içerisinde kullanılmaktadır:

p_x, p_y → oyuncunun konumu
s_x, b_x → engellerin konumu
j_state → zıplama durumu
skor → oyuncunun puanı

Örnek kod:

```
p_x DB 10
p_y DB 18
s_x DB 75
```

```
b_x DB 110
j_state DB 0
skor DB 0
```

4.2 Oyun Döngüsü

Oyun sürekli çalışan bir döngü yapısına sahiptir:

```
ANA_DONGU:

MOV CX, 0000h
MOV DX, 0300h
MOV AH, 86h
INT 15h
```

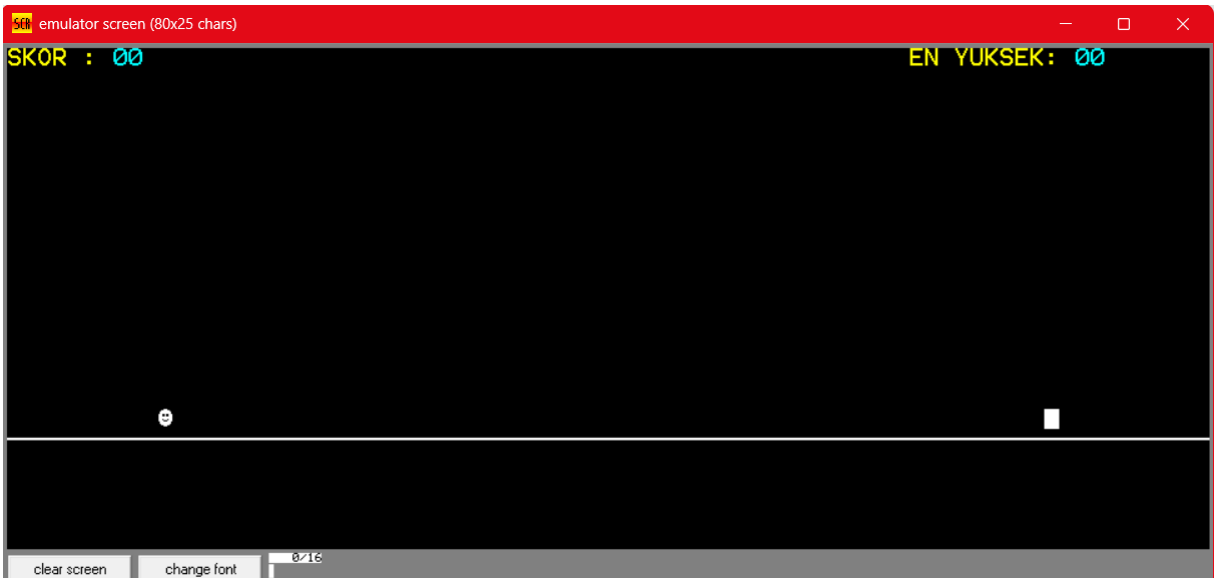
Bu yapı sayesinde oyun kontrollü hızda çalışmaktadır.

4.3 Klavye Girişi

Klavye kontrolü BIOS interrupt ile yapılmaktadır:

```
MOV AH, 01h
INT 16h
JZ TUS_YOK
```

Kullanıcı SPACE tuşuna bastığında karakter zıplamaktadır.



4.4 Zıplama Mekaniği

Zıplama işlemi bir durum (state) sistemi ile yapılmaktadır:

```
CMP j_state, 1
JE YUKARI
CMP j_state, 2
JE ASAGI
```

Karakter yukarı çıkarken ve aşağı inerken farklı işlemler uygulanmaktadır.

4.5 Engel Hareketi

Engeller sürekli sola doğru hareket etmektedir:

```
SUB s_x, 2
SUB b_x, 2
```

Ekran dışına çıktığında tekrar başa alınmaktadır.

4.6 Çarpışma Kontrolü

Karakter ile engel çarpışması şu şekilde kontrol edilir:

```
CMP AL, s_x
JE Y_BAK

CMP p_y, 18
JNE DEVAM
MOV over_f, 1
```

Çarpışma gerçekleştiğinde oyun sona ermektedir.

4.7 Skor Sistemi

Oyuncu engelleri geçtikçe skor artmaktadır:

```
INC skor
```

Ayrıca en yüksek skor da tutulmaktadır.

4.8 Ekrana Yazdırma

Ekrana yazdırma işlemi video belleği üzerinden yapılmaktadır:

```
MOV AX, 0B800h
MOV ES, AX
```

Bu yöntem hızlı ve düşük seviyeli bir ekran kontrolü sağlar.

4.9 Game Over

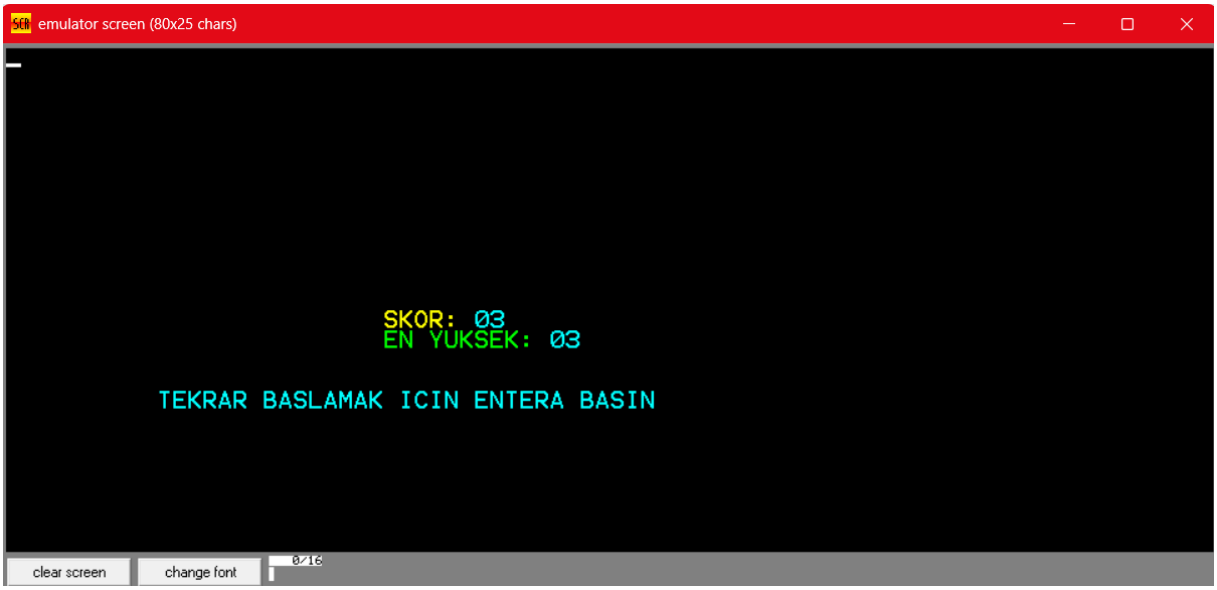
Oyun bitiş kontrolü:

```
CMP over_f, 1
JE GAME_OVER
```

Oyun bittikten sonra:

Final skor gösterilir

ENTER ile yeniden başlatılır



4.10 Prosedürler ve Fonksiyonel Yapı

Program, modülerliği sağlamak amacıyla farklı prosedürlere (PROC) ayrılmıştır. Bu sayede kod daha okunabilir ve yönetilebilir hale getirilmiştir.

4.10.1 Zıplama Kontrol Prosedürü

```
ZIPLAMA_KONTROL PROC
    CMP j_state, 1
    JE  YUKARI
    CMP j_state, 2
    JE  ASAGI
    RET

YUKARI:
    SUB p_y, 2
    CMP p_y, 14
    JNE Z_RET
    MOV p_y, 14
    MOV j_state, 2
    RET

ASAGI:
    ADD p_y, 2
    CMP p_y, 18
    JNE Z_RET
    MOV p_y, 18
    MOV j_state, 0

Z_RET:
    RET
ZIPLAMA_KONTROL ENDP
```

Açıklama:

- Karakterin zıplama hareketini kontrol eder
- Yukarı çıkma ve aşağı inme işlemleri ayrı ayrı yönetilir
- `j_state` değişkeni bir durum makinesi gibi çalışır

4.10.2 Skor Kontrol Prosedürü

```
SKOR_KONTROL_ET PROC

    MOV AL, s_pass
    CMP AL, 1
    JE  IKINCI_ENGEL

    MOV AL, s_x
    CMP AL, p_x
    JAE IKINCI_ENGEL
```

```
    INC skor

    MOV AL, skor
    CMP AL, en_yuksek_skor
    JBE REKOR_GECME_1
    MOV en_yuksek_skor, AL

REKOR_GECME_1:

    CALL SKOR_YAZ
    MOV s_pass, 1
```

Açıklama:

- Engel geçildiğinde skor artırılır
- Aynı engelden tekrar puan alınmasını engellemek için `s_pass` kullanılır
- En yüksek skor güncellenir

4.10.3 Çarpışma Kontrol Prosedürü

```
CARPISMA_KONTROL PROC
    MOV AL, p_x
    CMP AL, s_x
    JE  Y_BAK
    DEC AL
    CMP AL, s_x
    JE  Y_BAK

    MOV AL, p_x
    CMP AL, b_x
    JE  Y_BAK
    DEC AL
    CMP AL, b_x
    JE  Y_BAK
    RET

Y_BAK:
    MOV AL, p_y
    CMP AL, 18
    JNE CC_RET
    MOV over_f, 1

CC_RET:
    RET
CARPISMA_KONTROL ENDP
```

Açıklama:

- Karakter ile engellerin çarpışmasını kontrol eder
- X ve Y koordinatları karşılaştırılır
- Çarpışma durumunda oyun sonlandırılır

4.10.4 Ekran Çizim Prosedürleri

```
CIZIMLER PROC

    MOV DH, p_y
    MOV DL, p_x
    CALL IMLEC
    MOV AL, 02h
    CALL KARAKTER_BAS
ENGELLERI_CIZ PROC

    MOV DH, 18
    MOV DL, s_x
    CALL IMLEC
    MOV AL, 0DBh
    CALL KARAKTER_BAS
```

Açıklama:

- Oyuncu ve engeller ekrana çizilir
- `IMLEC` prosedürü imleci konumlandırır
- `KARAKTER_BAS` ekrana karakter basar

4.10.5 Ekrana Karakter Basma

```
KARAKTER_BAS PROC
    MOV AH, 09h
    MOV BH, 0
    MOV BL, 0Fh
    MOV CX, 1
    INT 10h
    RET
KARAKTER_BAS ENDP
```

Açıklama:

- BIOS interrupt (INT 10h) kullanılır
- Ekrana karakter yazdırılır
- Renk ve karakter kontrol edilir

4.10.6 İmleç Konumlandırma

```
IMLEC PROC
    MOV AH, 02h
    MOV BH, 0
```

```
    INT 10h
    RET
IMLEC ENDP
```

Açıklama:

- Ekranda imlecin konumunu belirler
- Karakter çizimi için gereklidir

4.10.7 Zemin Çizimi

```
ZEMIN_CIZ PROC
    MOV DH, 19
    MOV DL, 0

Z_L:
    CALL IMLEC
    MOV AL, 0C4h
    CALL KARAKTER_BAS
    INC DL
    CMP DL, 80
    JB  Z_L
    RET
ZEMIN_CIZ ENDP
```

Açıklama:

- Oyunun alt kısmına zemin çizer
- 80 sütun boyunca tekrar eder

5. BULGULAR

Proje çalıştırıldığında:

- Klavye girişlerinin doğru çalıştığı
- Karakterin düzgün zıpladığı
- Engellerin sürekli hareket ettiği
- Skorun doğru hesaplandığı
- Çarpışma durumunda oyunun sonlandığı

gözlemlenmiştir.

6. SONUÇ

Bu projede Assembly dili kullanılarak gerçek zamanlı çalışan bir oyun geliştirilmiştir. Proje sayesinde:

- Interrupt kullanımı
- Bellek yönetimi
- Oyun döngüsü mantığı

başarıyla uygulanmıştır.

7. ÖNERİLER

- Grafik moduna geçilebilir
- Daha fazla engel eklenebilir
- Ses sistemi eklenebilir
- Zorluk seviyesi artırılabilir